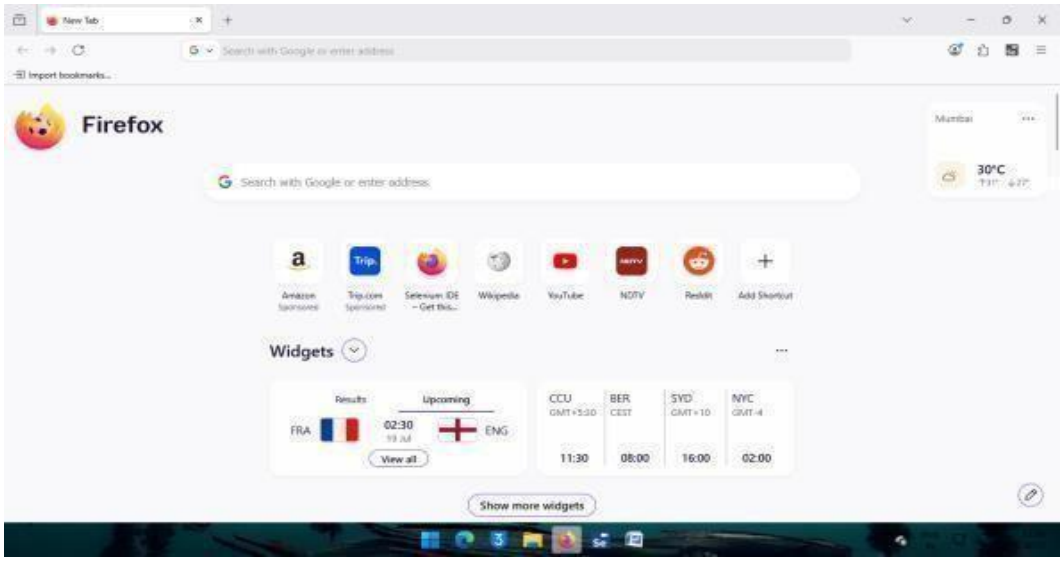
INDEX

SR	PRACTICALS	DATE	SIGN
1	To install Selenium IDE in web browser	18/07/2026	
2	To test demo website in Selenium IDE	18/07/2026	
3	To test login demo website in Selenium IDE	18/07/2026	
4	To create a test suite containing a minimum of 4 test cases to validate different web page formats (HTML page, XML response, JSON API output, and a dynamic form page), execute the test suite, and generate the report	25/07/2026	
5	To perform cross-website functional testing using selenium ide procedure	30/07/2026	
6	To Prepare Test Plan, Test Scenarios, Test Cases and Execute Test Cases Manually and prepare Test Execution Report and Defect Report as per STLC process.	22/08/2026	
7	To perform Selenium Server (Grid) Configuration and Remote Execution	12/09/2026	
8	To perform automation script of login on specified URL using login assertions	26/09/2026	
9	To perform Web Element Interaction and Synchronization Handling	26/09/2026	
10	Design and execute load testing scenarios for web applications and analyse response time.	07/10/2026	

PRACTICAL NO 1

AIM : To install Selenium IDE in web browser.

Step 1: Open a browser (Firefox, Edge)



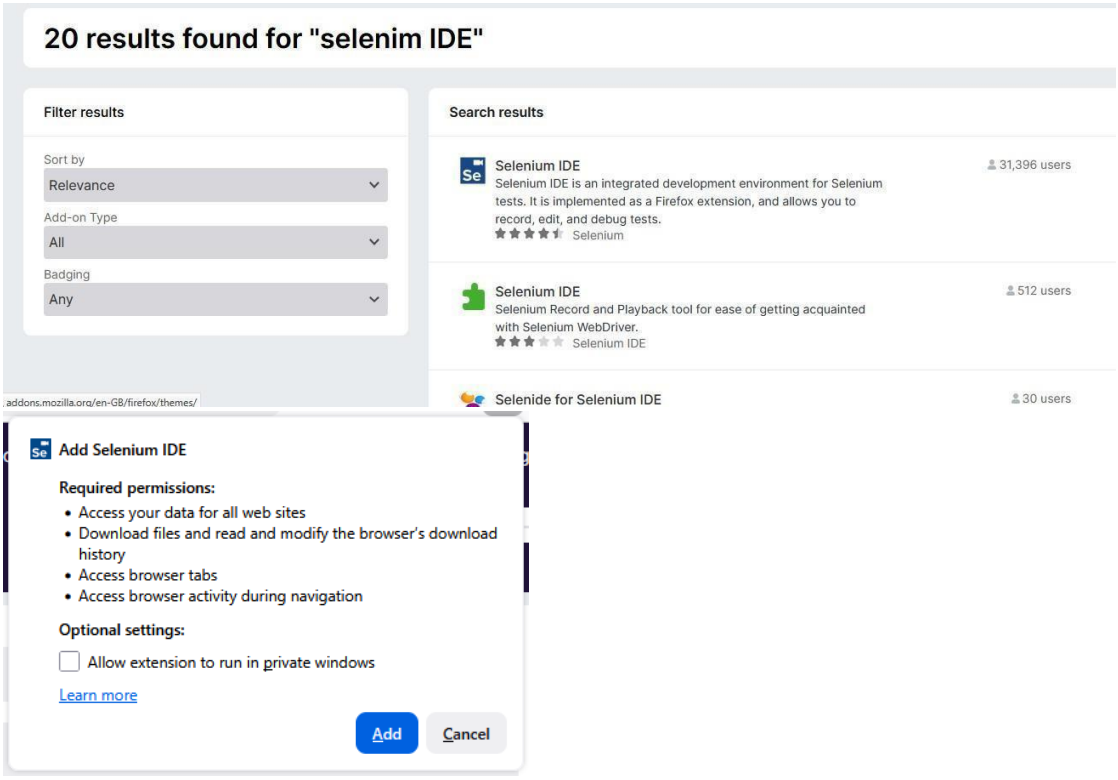


Step 2: Go to the right-hand corner.

Step 3: Click on Extensions & Themes.

Step 4: In the search bar, type Selenium IDE.

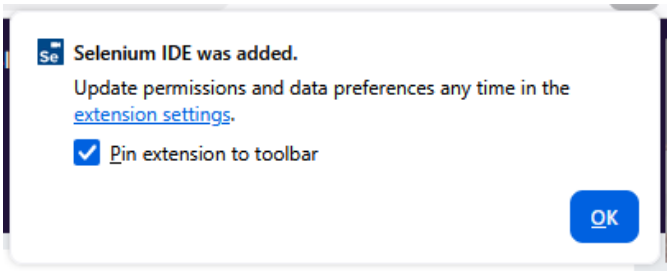
Step 5: Click on the Selenium IDE icon.



Step 6: Click Add to Firefox

Step 7: Click on Add to continue

Step 8: We can see Selenium IDE was added.



Step 8: We can see Selenium IDE was added.

Step 9: Click OK to continue.

Conclusion:

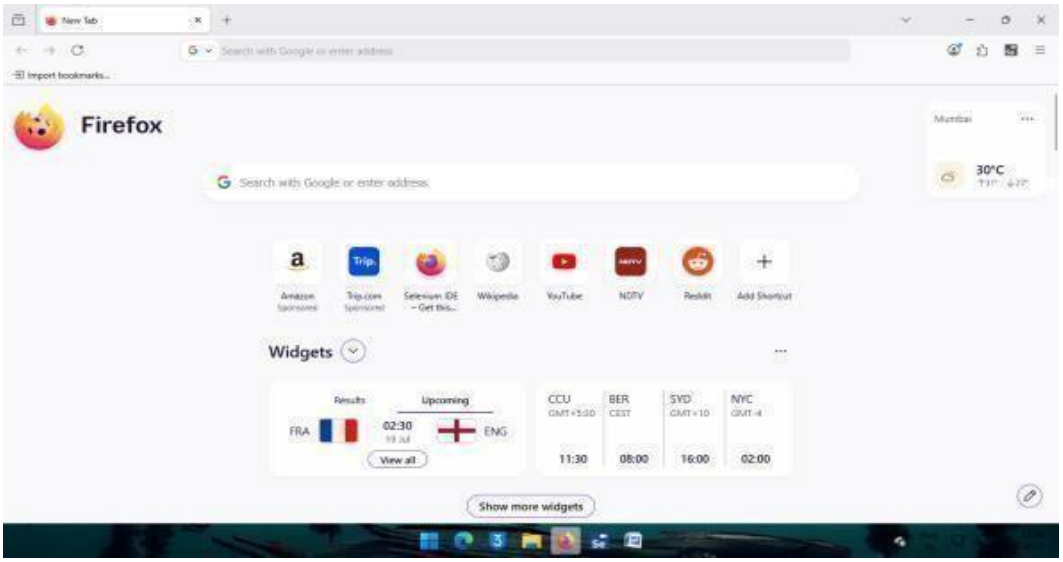
This practical helped us understand how to install and configure Selenium IDE as a browser extension. We learned the initial setup required for web automation testing and became familiar with the Selenium IDE interface, which is the first step toward creating and executing automated test cases.

PRACTICAL NO 2

AIM : To test demo website in Selenium IDE

Note : Perform in Firefox

Step-1: Open web browser.



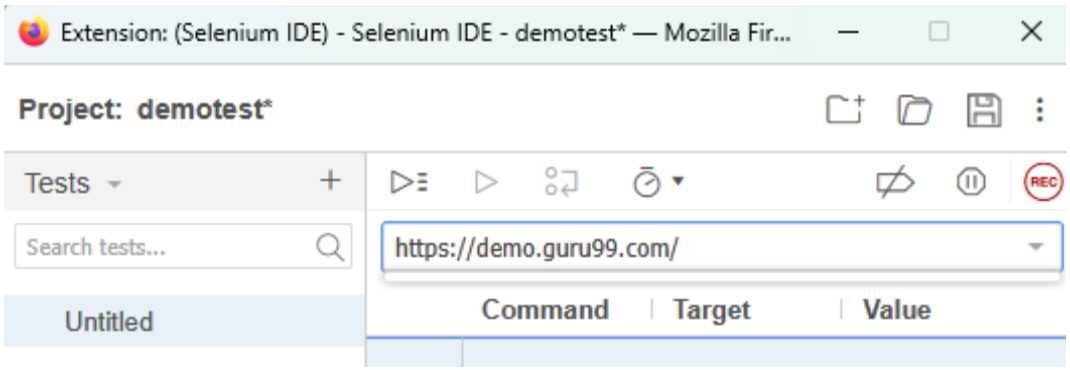
Step-2: Go to Selenium IDE extension.

Step-3: Click on Selenium IDE.

Step-4: Go to Create a New Project.

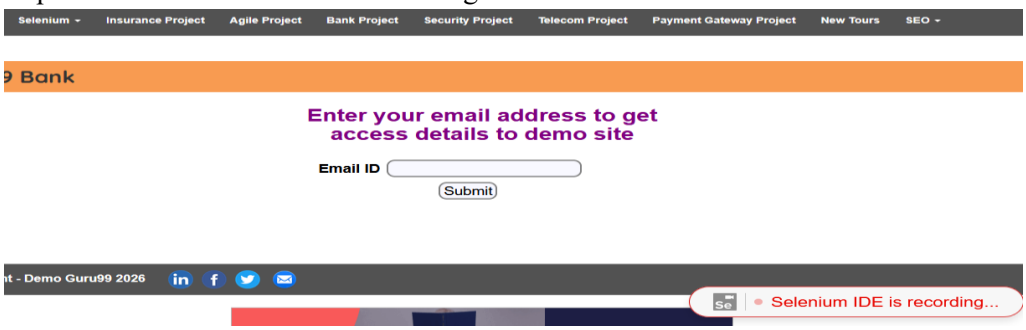
Step-5: Give project name as "demotest", click OK to continue.

Step-6: Paste a particular URL in the Selenium IDE.



Step-7: Click on Record.

Step-8: Selenium IDE will start recording.



Step-9: Right-click on any blank space on the web page.

Step-10: Go to Selenium IDE.

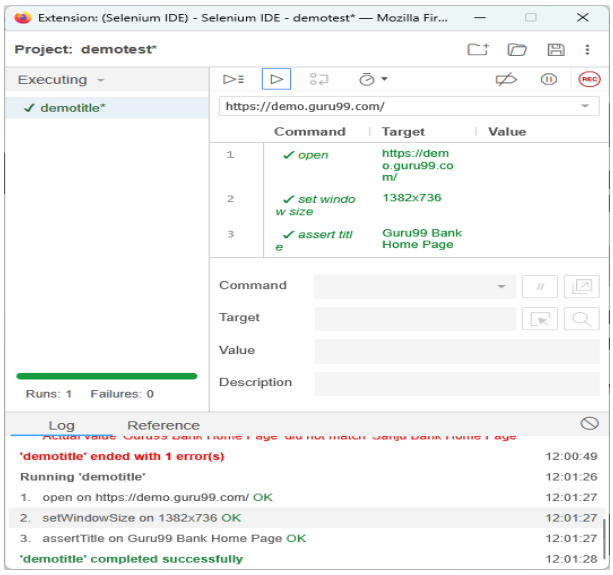
Step-11: Go to Assert.

Step-12: Go to Title and click.

Step-13: Come back again to Selenium IDE and stop recording.

Step-14: Give a test name as DemoTitle, click OK to continue.

Step-15: Click on DemoTitle



Conclusion:

This practical helped us learn the basics of automated web testing using Selenium IDE. We understood how to create a project, record user actions, use assertions (such as verifying the page title), save test cases, and execute them. This practical introduced us to the concept of validating web application behavior through automation instead of manual testing.

PRACTICAL NO 3

AIM : To test login demo website in Selenium IDE

Note : Perform in Edge

Step-1: Open the web browser.

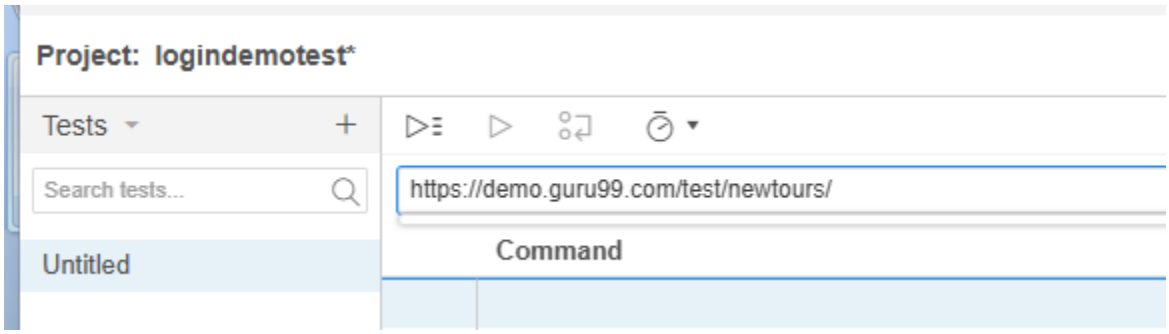
Step-2: Open the Selenium IDE extension.

Step-3: Click on Selenium IDE.

Step-4: Create a New Project.

Step-5: Give the project name as "LoginDemoTest" and click OK.

Step-6: Enter the demo website URL in Selenium IDE and click Start Recording.



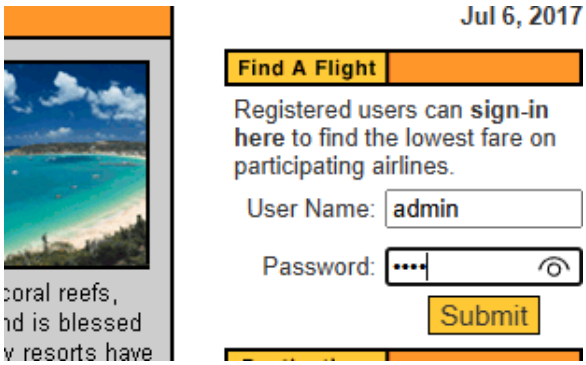
Step-7: On the login page, click on the Username field.

Step-8: Enter the valid username.

Step-9: Click on the Password field.

Step-10: Enter the valid password.

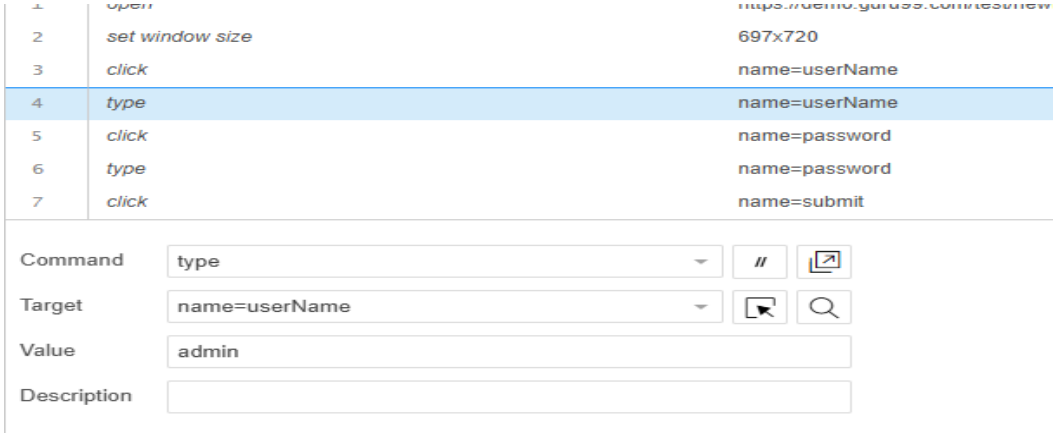
Step-11: Click on the Submit button

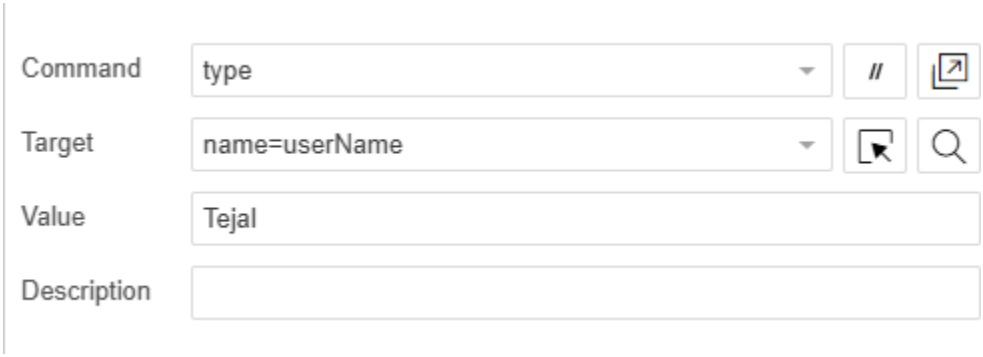


Step-12: Return to Selenium IDE and click Stop Recording.

Step-13: Save the test with the name LoginDemoTest and click OK.

Step-14: Click on the type option, once the information shows, change the name from admin to any other name,

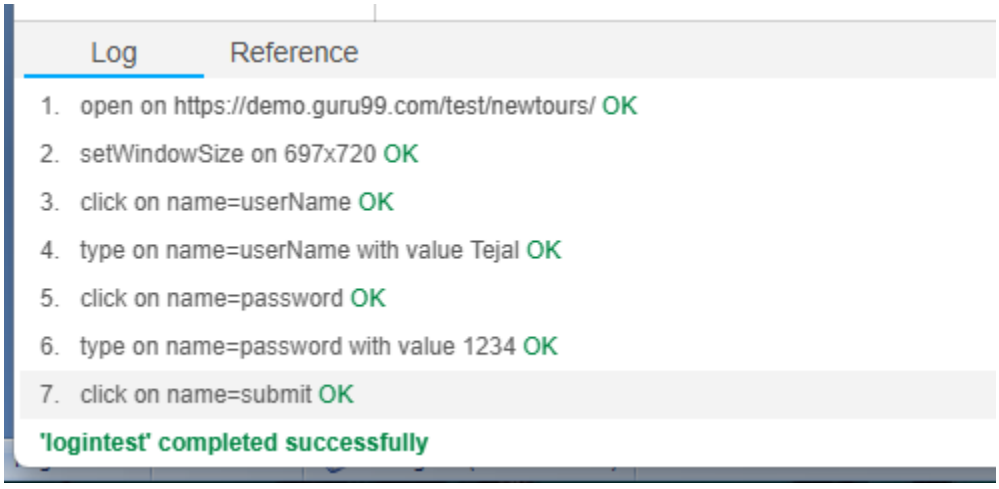




Step-15: Select LoginDemoTest from the project.

Step-15: Click on Run Current Test.

Step-16: Verify that the test passes successfully without any errors.



Conclusion:

1. This practical helped us understand how to automate the testing of a login functionality using Selenium IDE.

2. We learned to record user interactions, enter input into form fields, modify test data, execute the test case, and verify whether the login process behaves as expected.

3. This demonstrated the importance of automated functional testing for validating user authentication and improving software quality.

PRACTICAL NO 4

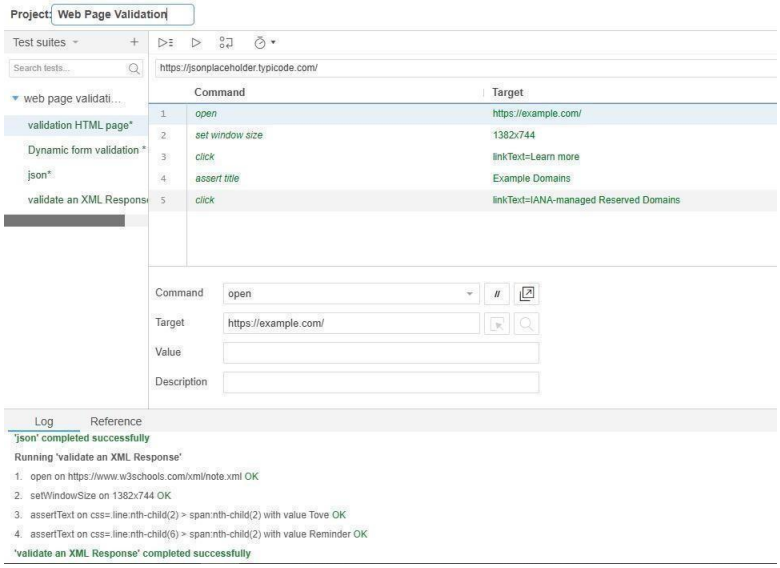
AIM: To create a test suite containing a minimum of 4 test cases to validate different web page formats (HTML page, XML response, JSON API output, and a dynamic form page), execute the test suite, and generate the report.

Step 1: Open Selenium IDE

1. Open Microsoft Edge.
2. Click on the Selenium IDE extension.
3. Create a project named WebPage Validation.

Step 2: Create Test Case 1 – Validate HTML Page

1. Click + New Test.
2. Name the test Validate an HTML Page.
3. Enter the base URL:https://example.com/
4. Record the required actions such as open, click, type, and assertText/assertTitle.
5. Save the test case

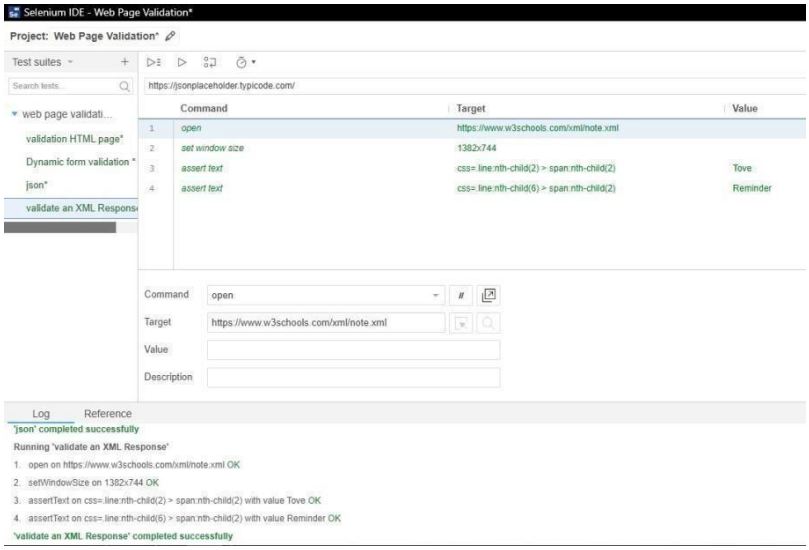


Step 3: Create Test Case 2 – Validate XML Response

1. Create another test named Validate an XML Response.
2. Open the URL: https://www.w3schools.com/xml/note.xml

3. Add commands like open, assertText, and verifyText.

4. Save the test case.



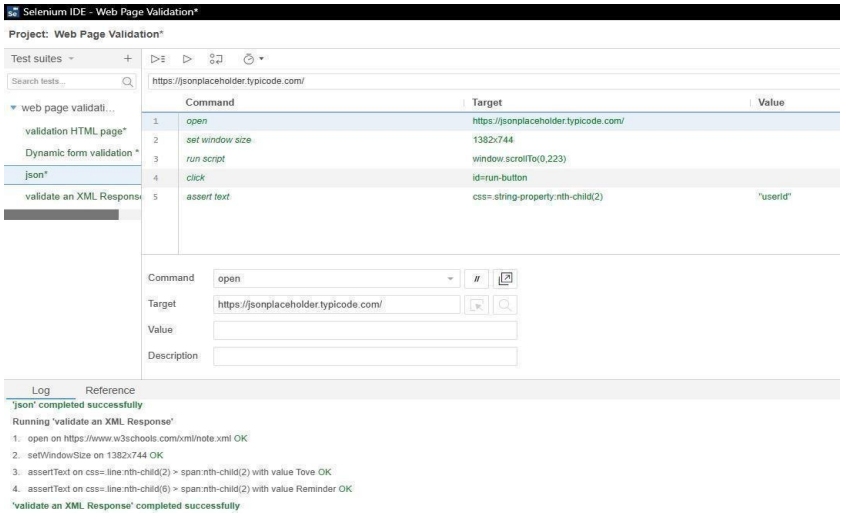
Step 4: Create Test Case 3 – Validate JSON API Output

1. Create a new test named JSON Validation.

2. Open the URL: https://jsonplaceholder.typicode.com/t

3. Add the commands open, assertText, and verifyText.

4. Save the test case.



Step 5: Create Test Case 4 – Validate Dynamic Form

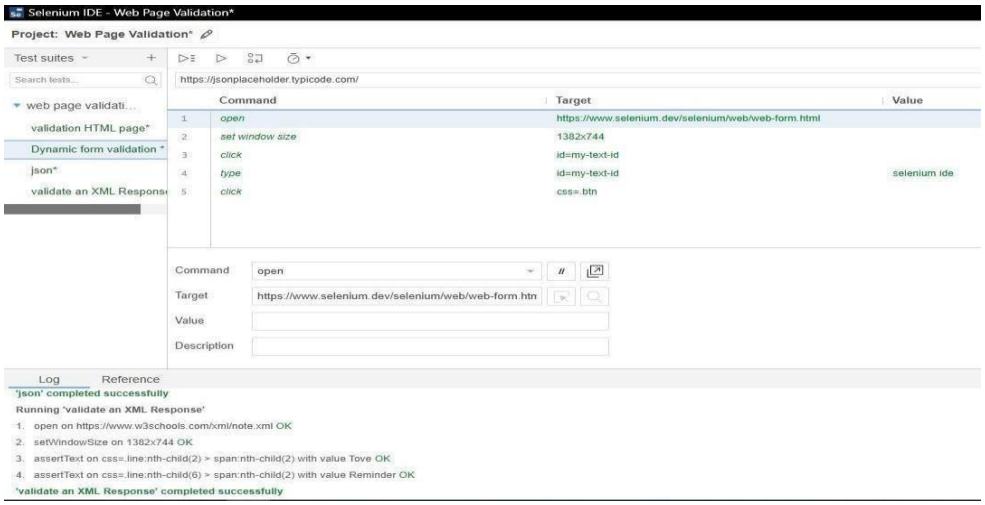
1. Create a new test named Dynamic Form Validation.

2. Open:https://www.selenium.dev/selenium/web/web-form.html

3. Record actions such as entering text, selecting a dropdown option, clicking a checkbox, and submitting the form

4. Add assertions to verify successful submission.

5. . Save the test case.



Step 6: Execute the Test Suite

1. Select the WebPage Validation Suite project.

2. Click the Run All Tests (►) button.

3. Observe the execution of all four test cases.

Step 7: Verify the Results

1. Check the Log section.

2. Ensure all commands show OK.

3. Verify that all test cases are Passed.

Step 8: Display the Report

1. View the execution results in the Log/Results panel.

2. Note the Passed, Failed, and Execution Status of the test suite.

3. Capture a screenshot of the successful execution report.

Conclusion:

1. This practical helped us understand the fundamentals of **web application testing and test automation** using Selenium IDE.
2. We learned how to create and organize multiple test cases into a test suite, automate browser interactions, validate different web page formats **(HTML, XML, JSON, and Dynamic web forms),** use assertions to verify expected results, and execute all test cases together. We also learned how to analyze execution logs and test reports to identify the success or failure of test cases.
3. These skills are essential for software testing, quality assurance (QA), and ensuring the reliability of web applications in real-world software development.

PRACTICAL NO 5

AIM : TO PERFORM CROSS-WEBSITE FUNCTIONAL TESTING USING SELENIUM IDE

PROCEDURE:

PREREQUISITES REQUIRE FOR PRACTICALS ARE MENTIONED BELOW

1 MICRO EDGE WEB BROWSER SELENIUM IDE WEB EXTENSION 2 DIFF OPEN SOURCE WEBSITES TO TEST MULTIPLE TESTCASES ACC.

Step 1: Open Selenium IDE

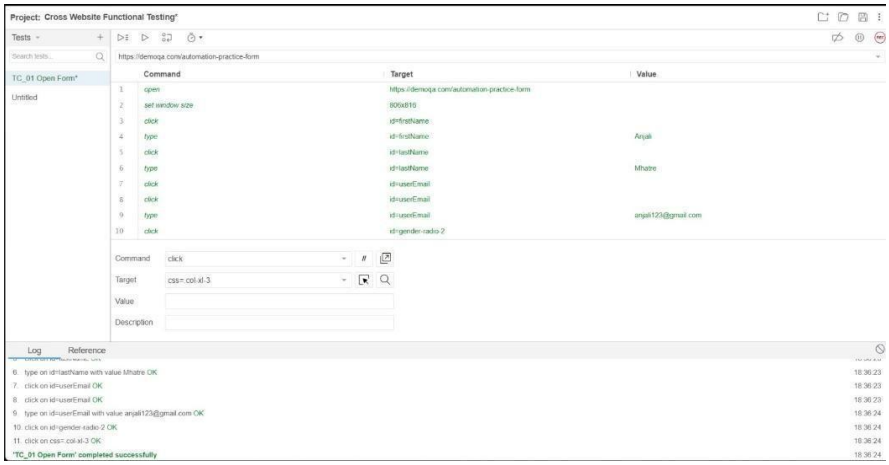
1. Launch **Microsoft Edge**.
2. Open the **Selenium IDE** extension.
3. Open the existing project or create a new project for cross-website functional testing.

Step 2: Record/Create Test Cases Test

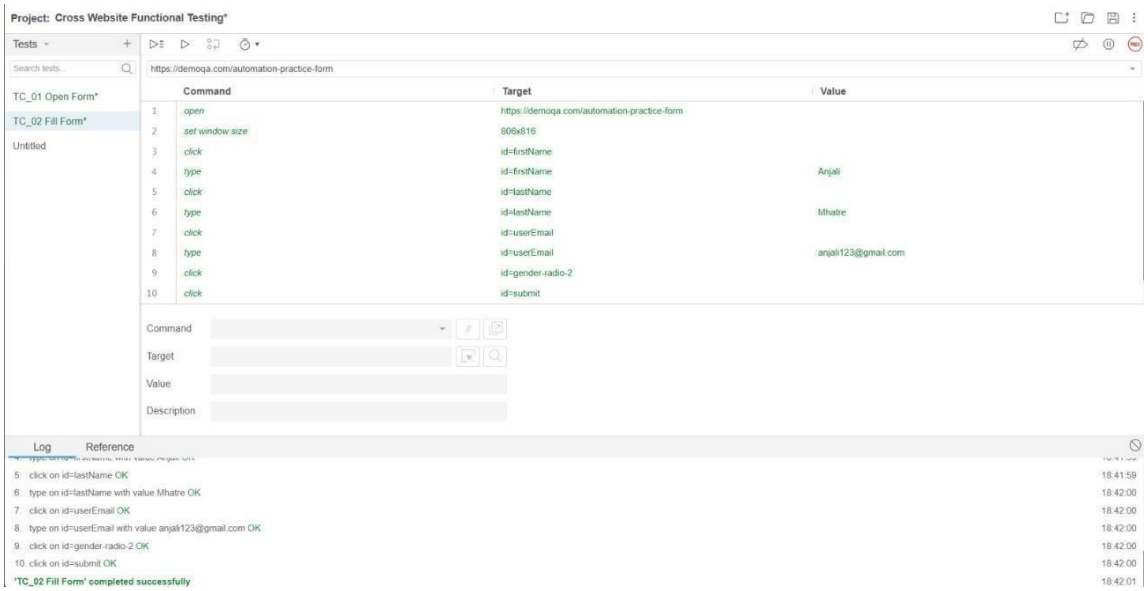
Case 1: Fill Form & Case 2: Fill Form

1. Open `https://demoqa.com/automation-practice-form`.
2. Set the browser window size.
3. Enter the First Name.
4. Enter the Last Name.
5. Enter the Email Address.
6. Select the Gender.
7. Enter the Mobile Number.
8. Enter the Subject (Maths).
9. Select the required Hobby.
10. Save the test case as **TC02_FILLFORM**.

Test Case 1: TC_01 OPEN FORM



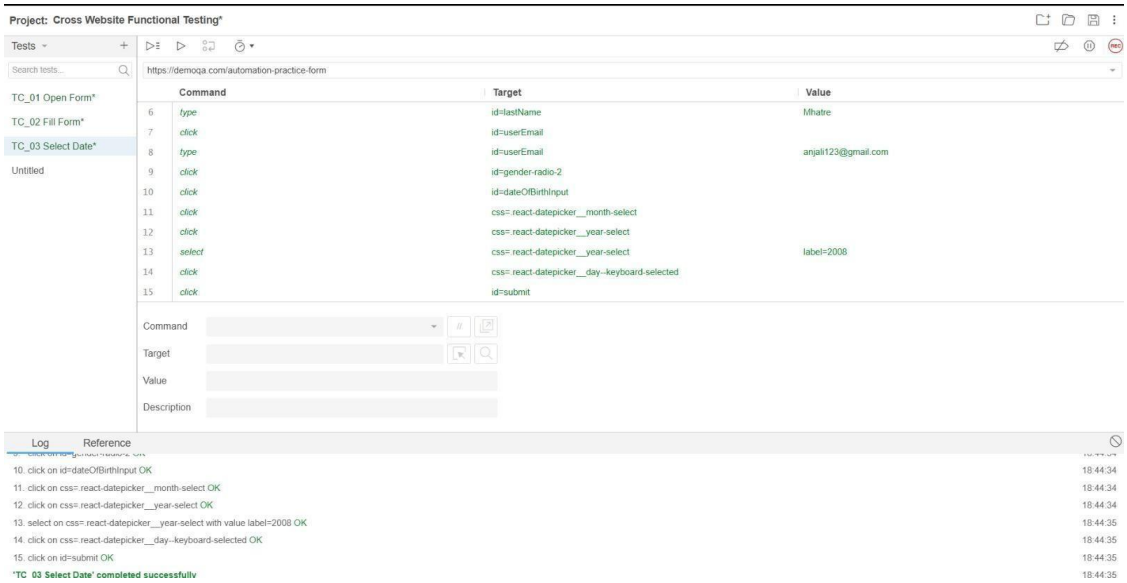
Test Case 2: TC_02 FILL FORM



Case 3: Select Date of Birth (DemoQA)

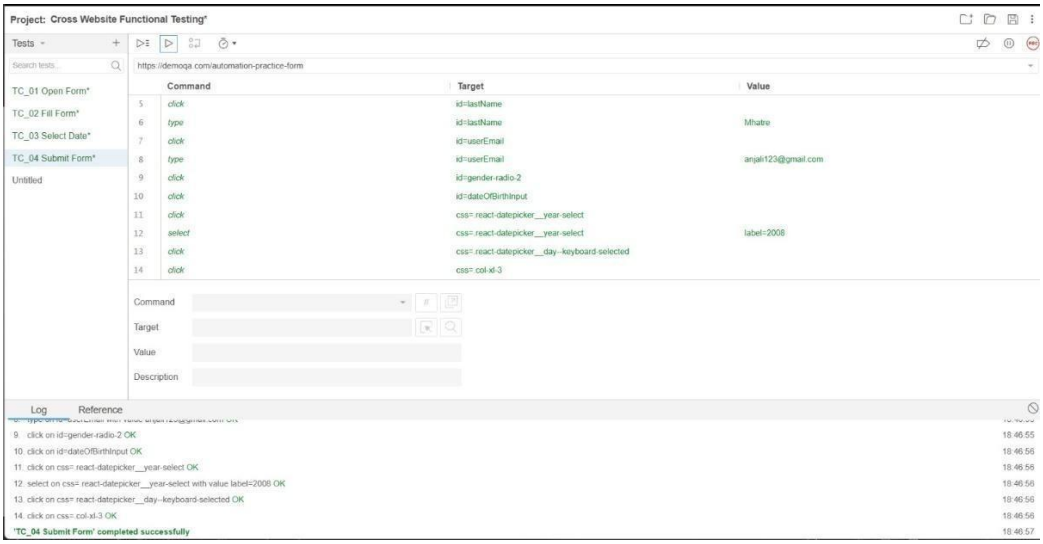
1. Open the DemoQA Practice Form.
2. Click the Date of Birth field.
3. Select the required Month.
4. Select the required Year.
5. Select the desired Date.
6. Save the test case as **TC03_SELECTDATE**.

Test Case 3: TC_03 SELECT DATE OF BIRTH



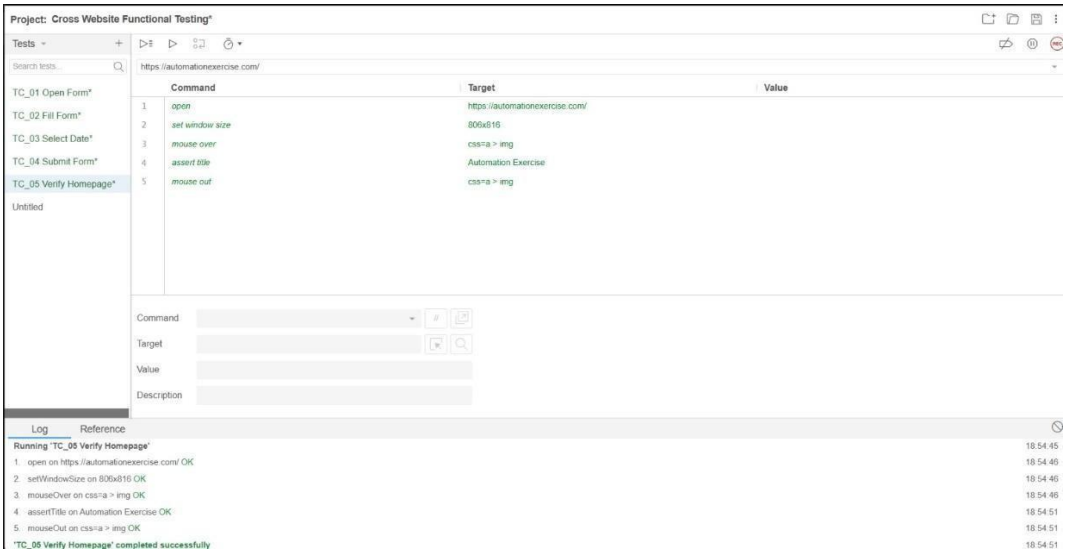
Case 4: SUBMIT FORM (DEMO QA)

1. Select a Hobby.
2. Click the **Submit** button.
3. Verify that the submission dialog appears.
4. Close the confirmation dialog.
5. Save the test case as **TC04_SUBMITFORM**.



Case 5: Verify HomePage

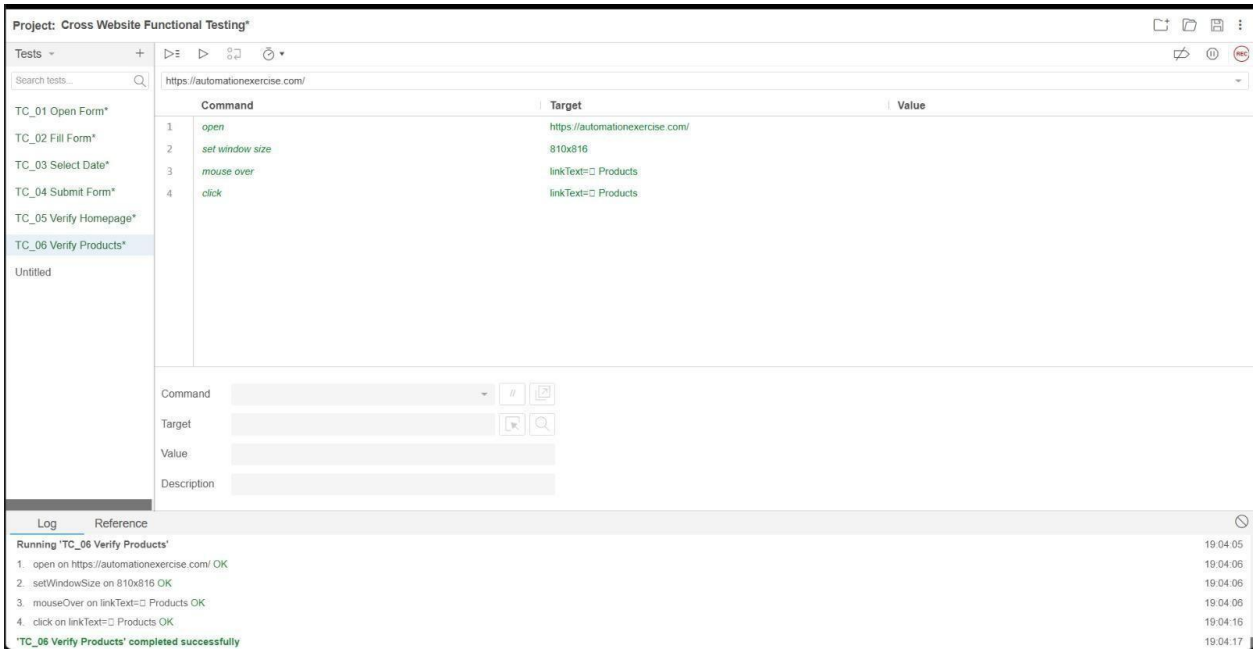
1. Open the Practice Form.
2. Enter valid details in all required fields.
3. Click the **Submit** button.
4. Verify that the form is submitted successfully.
5. Save the test case as **TC05_VERIFY_PAGE**



Case 6: **VERIFY PRODUCTS**

1. Open `https://automationexercise.com`.
2. Navigate to the **Products** page.
3. Enter **Men T-shirt** in the search box.
4. Click the **Search** button.
5. Open the product details page.
6. Verify that the product information is displayed correctly.
7. Save the test case as T**est Case 6: VERIFY_PRODUCTS**

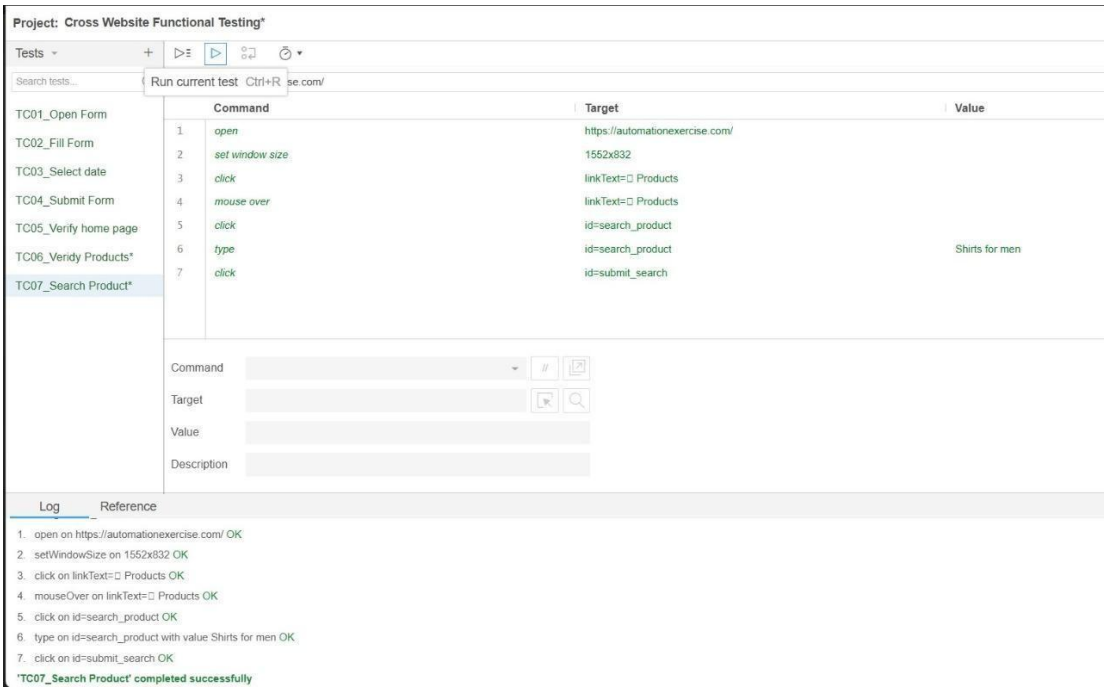
Test Case 6: VERIFY_PRODUCTS



Case 7: SEARCH PRODUCTS

1. Open `https://automationexercise.com`.
2. Navigate to the **Products** page.
3. Enter **Men T-shirt** in the search box.
4. Click the **Search** button
5. Search TShirt Button
6. Save the Test Case 7: SEARCH_ PRODUCTS

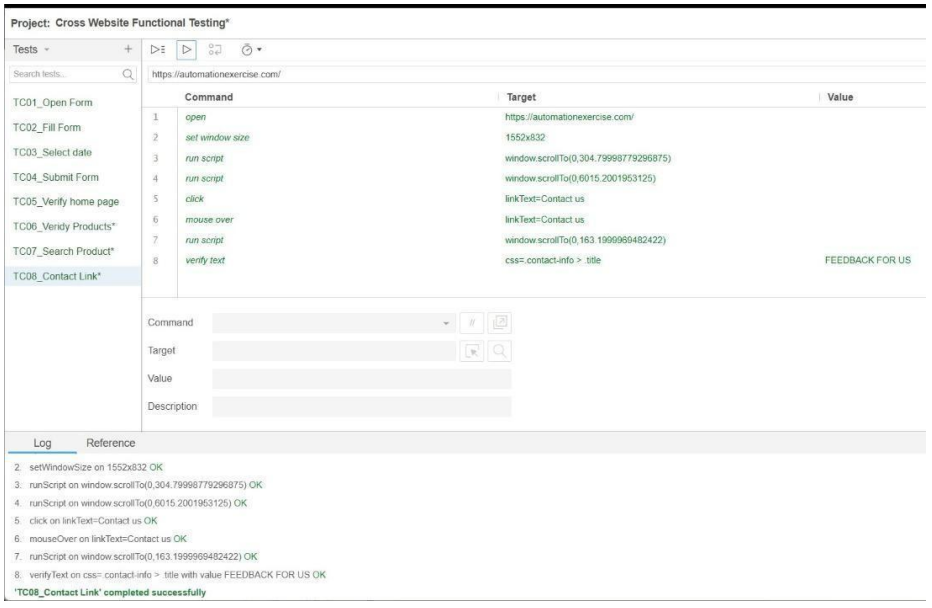
Test Case 7: SEARCH_PRODUCTS



Case 8: CONTACT LINK

1. Navigate to the **Products** page.
2. Click Navigation Page
3. Click the **Search** button
4. Click Contact links
5. Save Contact links Test Case 8: CONTACT_LINK

Test Case 8: CONTACT LINKS



INDEX

SR	PRACTICALS	DATE	SIGN
1	To install Selenium IDE in web browser	18/07/2026	
2	To test demo website in Selenium IDE	18/07/2026	
3	To test login demo website in Selenium IDE	18/07/2026	
4	To create a test suite containing a minimum of 4 test cases to validate different web page formats (HTML page, XML response, JSON API output, and a dynamic form page), execute the test suite, and generate the report	25/07/2026	
5	To perform cross-website functional testing using selenium ide procedure	30/07/2026	
6	To Prepare Test Plan, Test Scenarios, Test Cases and Execute Test Cases Manually and prepare Test Execution Report and Defect Report as per STLC process.	22/08/2026	
7	To perform Selenium Server (Grid) Configuration and Remote Execution	12/09/2026	
8	To perform automation script of login on specified URL using login assertions	26/09/2026	
9	To perform Web Element Interaction and Synchronization Handling	26/09/2026	
10	Design and execute load testing scenarios for web applications and analyse response time.	07/10/2026	

PRACTICAL 6

Aim: To Prepare Test Plan, Test Scenarios, Test Cases and Execute Test Cases Manually and prepare Test Execution Report and Defect Report as per STLC process.

Pre-requisites: Operating System: Windows 11**,** Browser: Google Chrome**,** Testing Method: Black Box Testing**,** Testing Type: Manual Functional Testing

Application: OrangeHRM Web Application

PROCEDURE:

1. Open Selenium IDE:

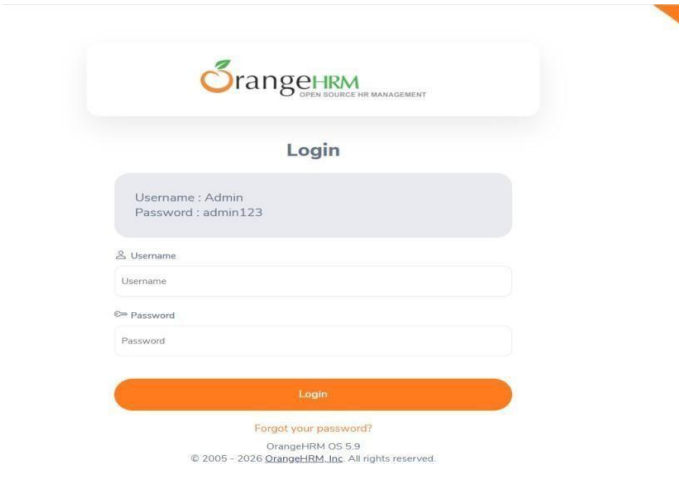
- Open Chrome.
- Launch the Selenium IDE extension.
- Create a new project/test case.
- Give the test an appropriate name, e.g. OrangeHRM_Login.

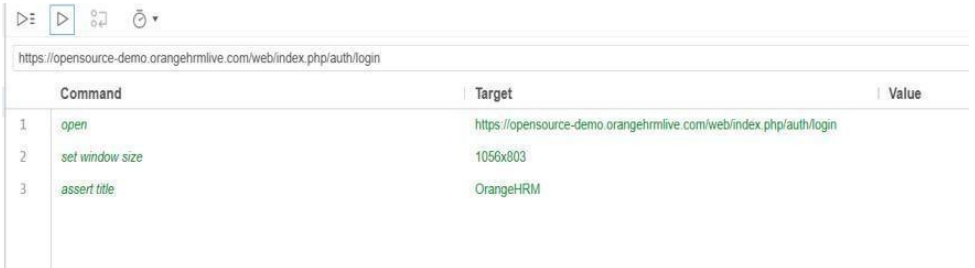
2. Test Case TC-001 — Verify Login Page

Record/execute the steps:

1. open → OrangeHRM URL: https://opensource-demo.orangehrmlive.com/web/index.php/auth/login
2. Verify Username field, password field, Login button, Forgot Password link
3. assert Text/assert Title

Expected output: Login page should be displayed



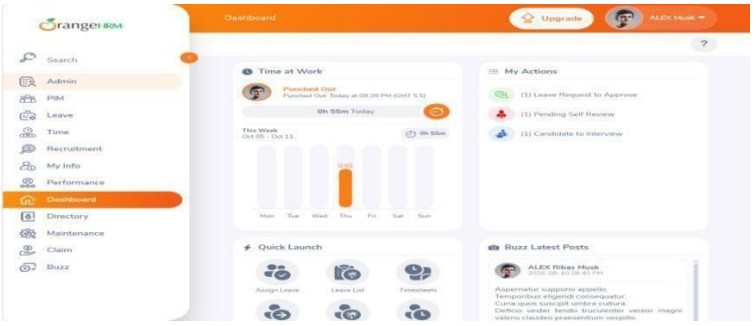


3. TC-002 — Valid Login

Add Selenium IDE commands corresponding to:

1. Open application.
2. **Enter username → Admin**
3. **Enter password → admin123**
4. Click Login.

Expected output: Dashboard should be displayed.



https://opensource-demo.orangehrmlive.com/web/index.php/auth/login

	Command	Target	Value
1	open	https://opensource-demo.orangehrmlive.com/web/index.php/auth/login	
2	set window size	1057x806	
3	type	name=username	Admin
4	click	name=password	
5	type	name=password	admin123
6	click	css=.oxd-button	

4. TC-003 — Invalid Username

Commands:

1. Open application.
2. **Enter username → WrongUser**
3. Enter password → admin123
4. Click Login.
5. Verify the error message.

Expected: Login should fail and an appropriate error message should appear.



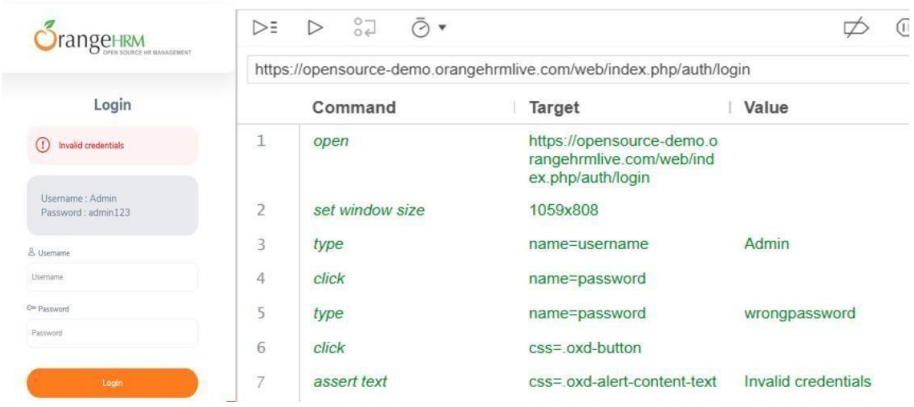
https://opensource-demo.orangehrmlive.com/web/index.php/auth/login

	Command	Target	Value
1	open	https://opensource-demo.orangehrmlive.com/web/index.php/auth/login	
2	set window size	1057x806	
3	type	name=username	Tejal
4	click	name=password	
5	type	name=password	admin123
6	click	css=.oxd-button	
7	assert text	css=.oxd-alert-content	Invalid credentials
8	click	css=.orangehrm-login-layout-blob	

5. TC-003 — Invalid Username

1. Open application.
2. Enter username → Admin
3. **Enter password → wrongpassword**
4. Click Login.
5. Verify the error message.

Expected: Login should fail and an appropriate error message should appear.

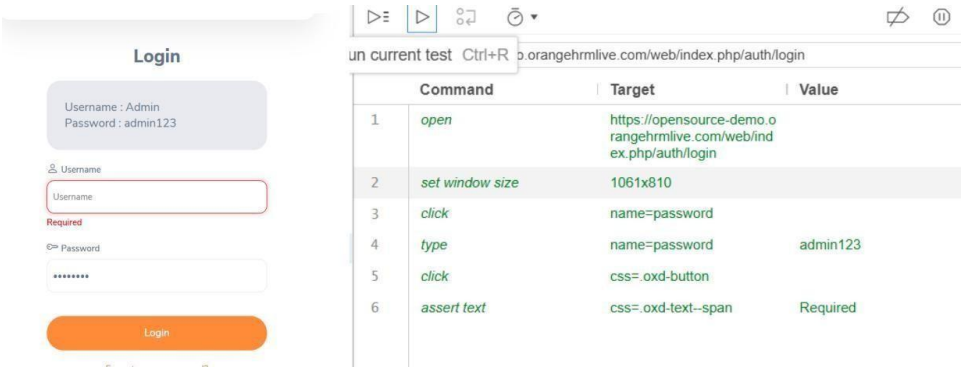


6. TC-005 — Blank Username

Commands:

1. Open application.
2. **Leave Username blank.**
3. Enter Password → admin123
4. Click Login.
5. Check username validation.

Expected: Username validation/required message should appear.

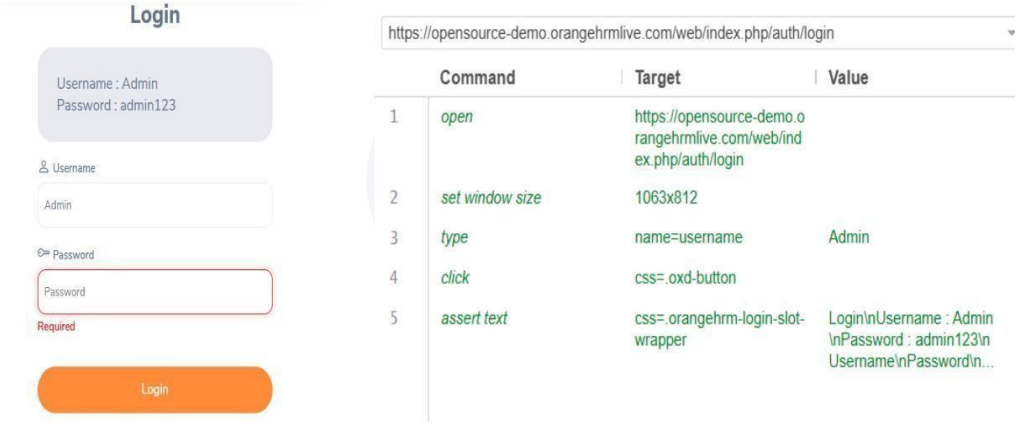


7. TC-006 — Blank Password:

1. Open application.
2. Username → Admin
3. **Leave Password blank.**

4. Click Login.
5. Verify password validation.

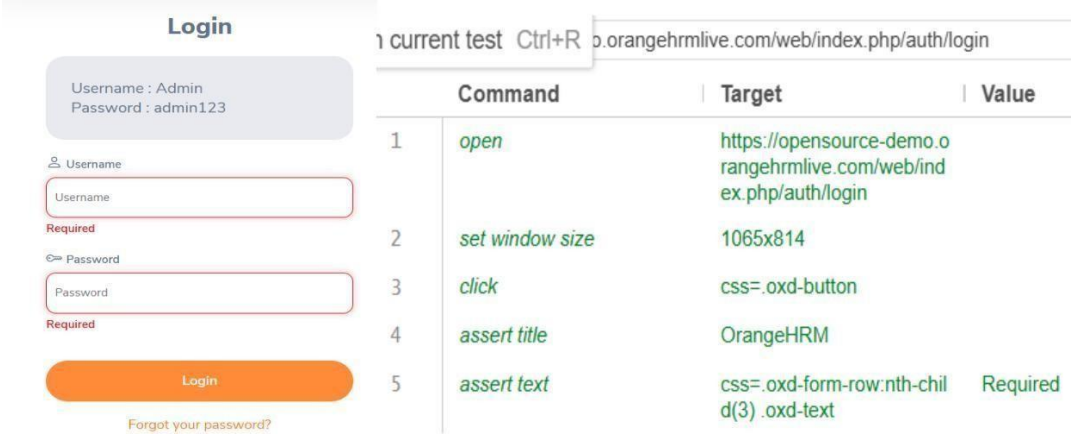
Expected: Password validation/required message should appear.



8. TC-007 — Both Fields Blank

1. Open application.
2. **Leave Username blank.**
3. **Leave Password blank**.
4. Click Login.
5. Verify validation messages.

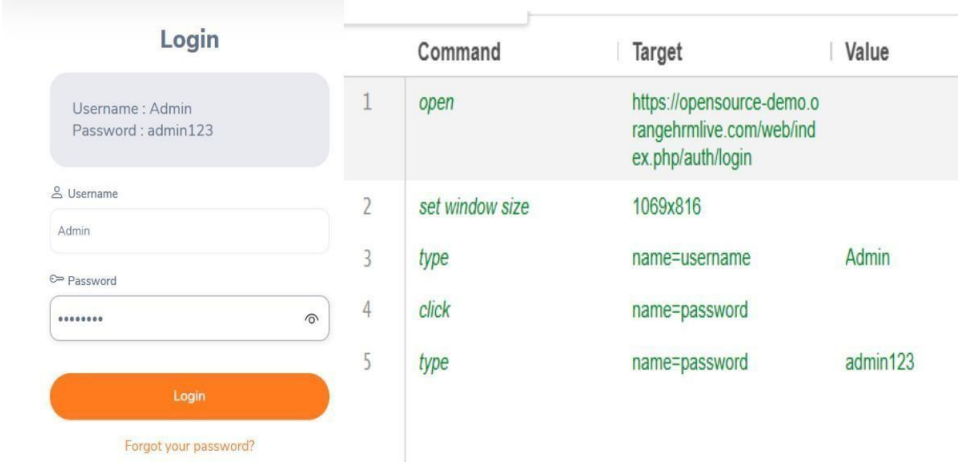
Expected: Validation messages for both Username and Password.



9. TC-008 — Password Masking

1. Open application.
2. Click Password field.
3. Type admin123.
4. Verify that the password is masked.

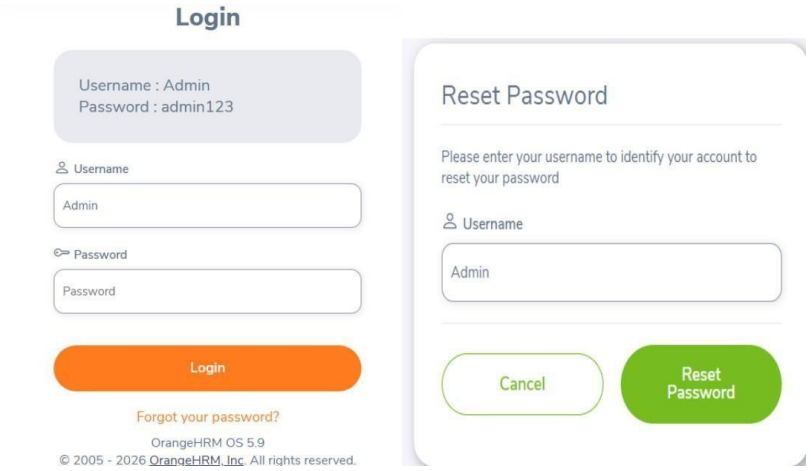
Expected: Password should appear as dots/asterisks.



10. TC-009 — Forgot Password

1. Open application.
2. Click Forgot Password.
3. Verify that the password recovery page opens.

Expected: Password recovery page should be displayed.



CONCLUSION:

The testing activity has been successfully completed**.**

11. Final Result

Finally, prepare the closure:

Total Test Cases: 9
Executed: 9
Passed after retesting: 9
Open Failed: 0

Testing Status: Completed.

PRACTICAL 7

Aim: Selenium Server (Grid) Configuration and Remote Execution

Step 1:-Install java

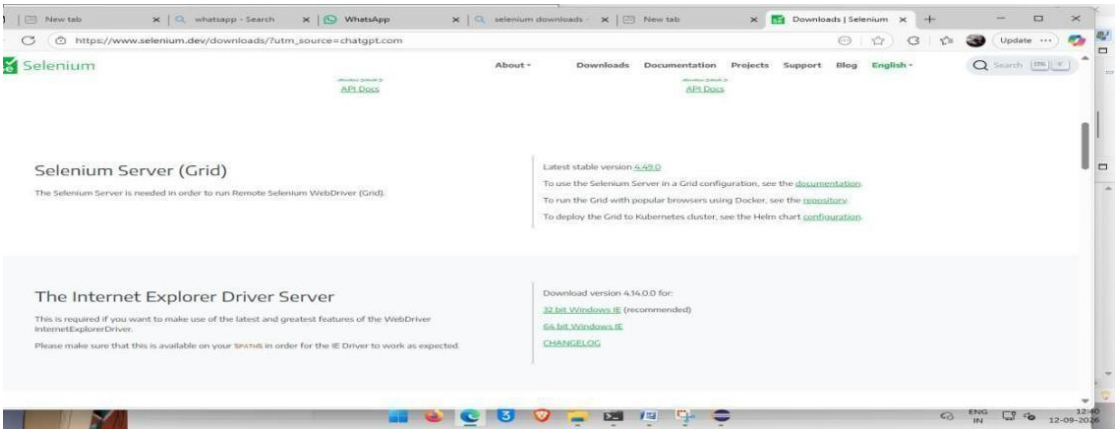
Check for version in command promt Cmd

java –version

Cmd javac –version

Step 2:-Install Selenium grid server latest version

Download Selenium ServerGo to the official Selenium website:Selenium Downloads



Find **Selenium Server** and download the .jar file.

For example:

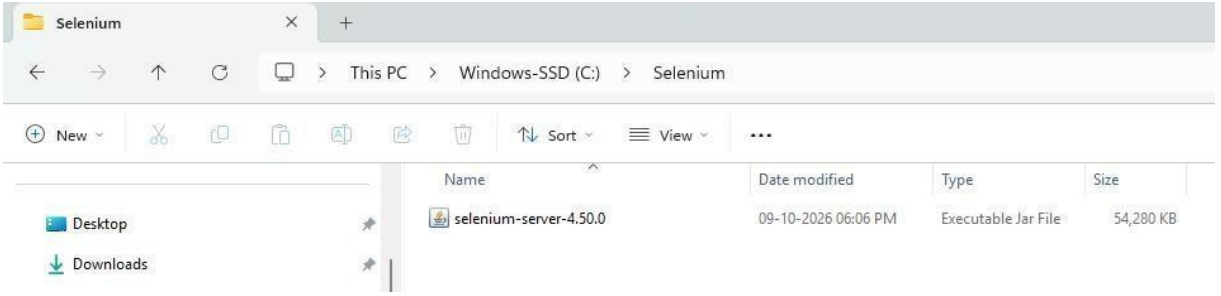
selenium-server-4.x.x.jar

Step 3: Create Selenium folder in C drive

Create:

C:\Selenium

Put the downloaded .jar file inside it.



For example:

C:\Selenium\selenium-server-4.x.x.jar

folder should look like:

C:
└── Selenium
    └── selenium-server-4.x.x.jar

```
C:\>cd Selenium

C:\Selenium>java -jar selenium-server-4.50.0.jar
Selenium Server commands

A list of all the commands available. To use one, run `java -jar selenium.jar
commandName`.
```

Step 4:-Start Selenium hub

Go to cmd and move to Selenium directory
Type java -jar selenium-server-4.x.x.jar hub(give proper version)

```
C:\Users\spaan>cd/

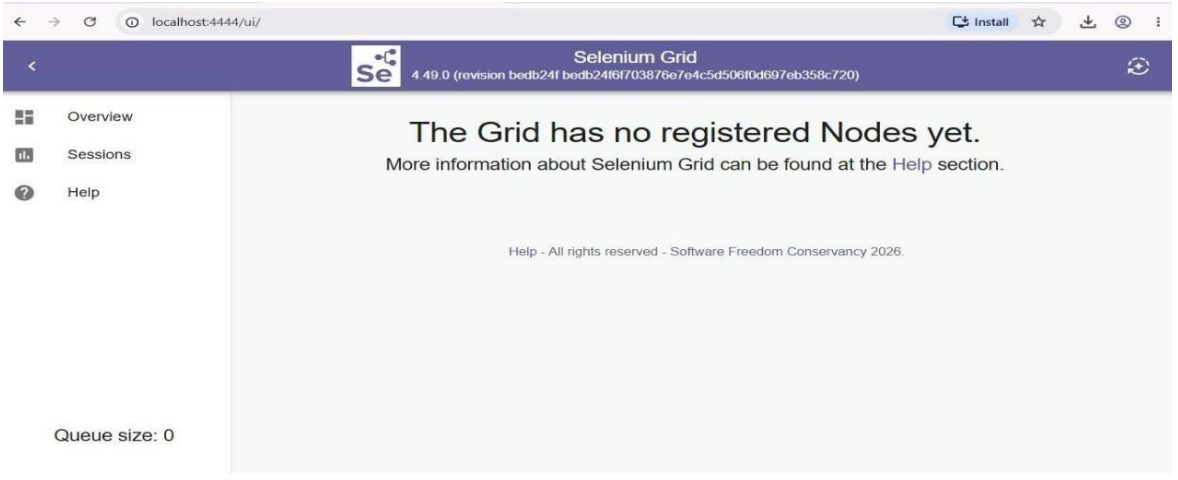
C:\>cd Selenium

C:\Selenium>java -jar selenium-server-4.50.0.jar hub
18:42:09.844 INFO [LoggingOptions.configureLogEncoding] - Using the system default encoding
18:42:09.853 INFO [OpenTelemetryTracer.createTracer] - Using OpenTelemetry for tracing
18:42:10.320 INFO [BoundZmqEventBus.<init>] - XPUB binding to [binding to tcp://*:4442, adve
```

Selenium Grid hub opens

Step 5:-

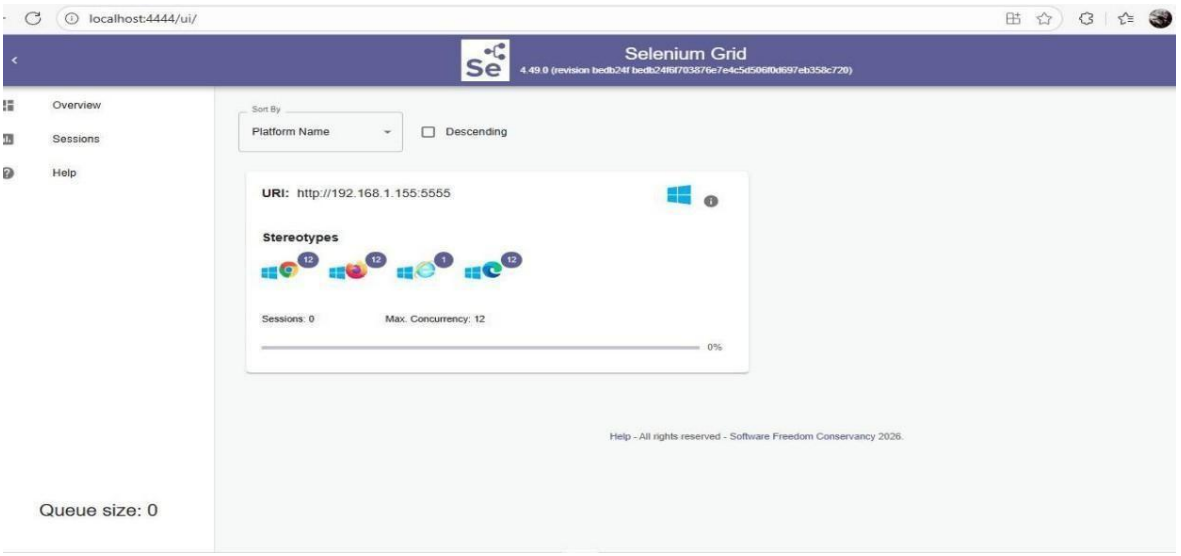
Open Chrome or Edge.Go to:http://localhost:4444 We should see the Selenium Grid page.



Step 6:-Start a node

Open other cmd go to selenium folder Type the following command

java -jar selenium-server-4.x.x.jar node --hub http://localhost:4444(give proper version)



Step 7:- open grid console

Verify whether all browsers are registered or not

Step 8 Create Maven Project

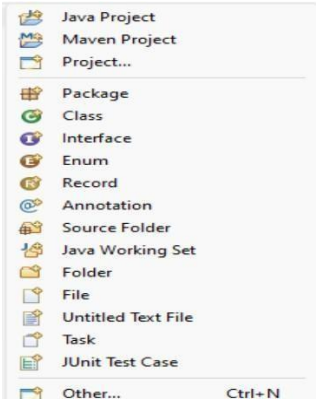
in Eclipse Step 8: Open

Eclipse

Open Eclipse.

Select:

File
↓
New
↓
Maven Project



Step 9: Select Maven project option

You will see:
Create a simple project (skip archetype selection)Check it. Click:**Next**

Step 10 — Fill Maven

Project Details Step 10:

Enter project information

Fill: Group

Id:com.selenium.grid

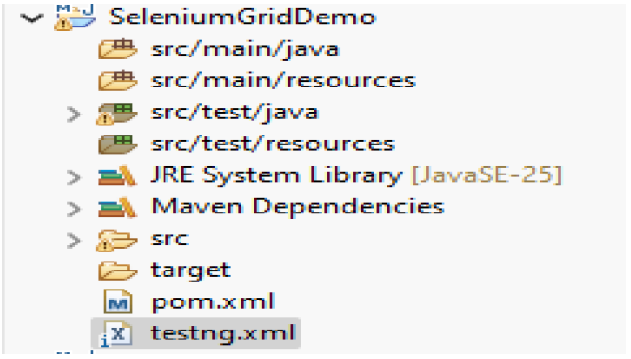
Artifact

Id:SeleniumGridDemo

Version:
0.0.1-SNAPSHOT

Packaging:
Jar

We can leave the version as the default.Click:**Finish**



Step 11 — Configure

pom.xml Step 11:

Open pom.xml

In Eclipse:

SeleniumGridDemo
└── pom.xml
Open it.Add Selenium and TestNG dependencies inside <dependencies>.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.selenium.grid</groupId>
<artifactId>SeleniumGridDemo</artifactId>
<version>0.0.1-SNAPSHOT</version>
<dependencies>

<!-- Selenium WebDriver -->
<dependency>
<groupId>org.seleniumhq.selenium</groupId>
<artifactId>selenium-java</artifactId>
<version>4.34.0</version>
</dependency>

<!-- TestNG -->
<dependency>
<groupId>org.testng</groupId>
<artifactId>testng</artifactId>
<version>7.11.0</version>
<scope>test</scope>
</dependency>

</dependencies>
  </project> Save
```

the file

Step 12 — Update Maven Project Step 12: Update dependencies

Right-click the project:SeleniumGridDemo

Select:Maven

↓

Update Project

Click:**OK**

Wait for Eclipse to download the dependencies

Step 13 — Create Package Step 13:

Create Java package

Go to:src/test/javaRight-click:

New
↓
Package

Enter:**com.selenium.grid**Click **Finish**.

Step 14 — Create Test Class Step 14: Create Java class

Right-click:**com.selenium.grid** AND Select:

New
↓
Class

Enter:**GridCrossBrowserTest**

Click **Finish**.

Step 15 — Write RemoteWebDriver Code Step 15: Add imports

```
packagecom.selenium.grid;
importjava.net.URI;  import
java.net.URL;
importorg.openqa.selenium.By;
importorg.openqa.selenium.WebDriver;
importorg.openqa.selenium.chrome.ChromeOptions;
importorg.openqa.selenium.firefox.FirefoxOptions;
importorg.openqa.selenium.remote.RemoteWebDriver;
importorg.testng.Assert;
importorg.testng.annotations.AfterMethod;
importorg.testng.annotations.BeforeMethod;
importorg.testng.annotations.Parameters;
importorg.testng.annotations.Test;
publicclassGridCrossBrowserTest {

    WebDriver driver;
    @BeforeMethod
    @Parameters("browser")
```

```
publicvoidsetup(String browser) throws Exception {

        URL gridUrl = URI.create("http://localhost:4444").toURL(); if

(browser.equalsIgnoreCase("chrome")) {
ChromeOptions options = newChromeOptions();

            driver = newRemoteWebDriver(
gridUrl,
                        options
                );
           } elseif (browser.equalsIgnoreCase("firefox")) {

FirefoxOptions options = newFirefoxOptions();
```

PRACTICAL 8

Aim: To perform automation script of login on specified URL using login assertions

1. Objective

Automate the login process of a web application and verify that login is successful using:

1. Username and password entry
2. Login button click
3. **URL validation**
4. **Welcome message validation**
5. **Logout button visibility validation**
6. Logout after successful login

2. Software Required

Install:

- Java JDK 17 or later
- Eclipse IDE
- Maven
- Google Chrome
- Selenium WebDriver
- TestNG

You do **not normally need to manually download ChromeDriver** with recent Selenium versions because Selenium Manager can automatically manage the browser driver.

3. Create Maven Project

Open Eclipse.

Step 1

Go to:

File → New → Maven Project

Step 2

Select:

Create a simple project (skip archetype selection)

Click **Next**.

Step 3 – Fill Maven Project Details

Use:

Field	Value
Group Id	com.demo
Artifact Id	LoginAutomation
Version	0.0.1-SNAPSHOT
Packaging	jar

Click **Finish**.

1. Project Structure

Your project should look approximately like this:

LoginAutomation

```
├── src/main/java
├── src/test/java
│   └── com.demo.login
│       └── LoginTest.java
├── pom.xml
└── JRE System Library
```

We will put our TestNG test class inside:
src/test/java

2. Add Selenium and TestNG Dependencies

Open: Copy pom.xml code

```
<?xml version="1.0" encoding="UTF-8"?>

  <project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>LoginAutomation</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <dependencies>

      <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>4.34.0</version>
      </dependency>
```

```
        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
            <version>7.11.0</version>
            <scope>test</scope>
        </dependency>

    </dependencies>
</project>
```

Save the file.

Maven should download the required Selenium and TestNG libraries.

3. Create TestNG Class

Right-click:

src/test/java

Select:

New → Package

Enter: com.demo.login

Then right-click the package:

New → Class

Class name:

LoginTest

Click **Finish**.

4. Complete Login Automation Code

Copy java class code: package
com.demo.login; import
java.time.Duration; import
org.openqa.selenium.By;
import org.openqa.selenium.WebDriver; import
org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions; import
org.openqa.selenium.support.ui.WebDriverWait; import
org.testng.Assert;
import org.testng.annotations.AfterMethod;

```
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
  public class LoginTest {
      WebDriver driver;
WebDriverWait wait;
      // Method to pause the browser
      public void pause() {
       try {
Thread.sleep(3000);
       } catch (InterruptedException e) {
Thread.currentThread().interrupt();
          }
      }
      @BeforeMethod
      public void setUp() {
  System.out.println("Starting browser...");
          driver = new ChromeDriver();
driver.manage().window().maximize();
pause();
System.out.println("Opening OrangeHRM...");
driver.get("https://opensource-demo.orangehrmlive.com/");
       wait = new WebDriverWait(driver, Duration.ofSeconds(15)); pause();
System.out.println("Login page opened.");
      }
      @Test
      public void loginValidation() {
       // STEP 1 - Username System.out.println("STEP 1:
Entering username..."); WebElement username =
wait.until( ExpectedConditions.visibilityOfElementLocated(
                        By.name("username")
                )
          );
username.sendKeys("Admin");
System.out.println("Username entered: Admin");
pause();
       // STEP 2 - Password System.out.println("STEP 2:
Entering password..."); WebElement password =
wait.until(
ExpectedConditions.visibilityOfElementLocated(
                        By.name("password")
                )
          );
password.sendKeys("admin123");
System.out.println("Password entered.");
pause();
System.out.println("STEP 3: Clicking Login button...");
WebElementloginButton = wait.until(
```

```
ExpectedConditions.elementToBeClickable(
By.xpath("//button[@type='submit']")
                )
        );
loginButton.click(); System.out.println("Login
button clicked."); pause();
        // STEP 4 - Dashboard
System.out.println("STEP 4: Waiting for Dashboard...");
wait.until( ExpectedConditions.urlContains("/dashboard")
        );
System.out.println("Dashboard loaded.");
pause();
        // STEP 5 - URL validation
  System.out.println("STEP 5: Checking Dashboard URL...");
        String actualURL = driver.getCurrentUrl();
System.out.println("Current URL: " + actualURL);
Assert.assertTrue( actualURL.contains("/dashboard"),
        );
System.out.println("PASS: URL validation successful."); pause();
        // STEP 6 - Dashboard text
System.out.println("STEP 6: Checking Dashboard text...");
WebElementdashboardText = wait.until(
ExpectedConditions.visibilityOfElementLocated(
By.xpath("//h6[text()='Dashboard']")
                )
        );
Assert.assertTrue(
dashboardText.isDisplayed(),
                "Dashboard is not displayed."
        );
Assert.assertEquals(
dashboardText.getText(),
                "Dashboard"
        );
System.out.println(
                "PASS: Dashboard text = "
                        + dashboardText.getText()
        );
pause();
       // STEP 7 - Profile menu System.out.println("STEP 7:
Opening profile menu..."); WebElementprofileMenu =
wait.until( ExpectedConditions.elementToBeClickable(
By.xpath(
                                "//span[contains(@class,'oxd-userdropdown-tab')]"
```

```
                    )
                )
        );
profileMenu.click(); System.out.println("Profile
menu opened."); pause();
        // STEP 8 - Logout
System.out.println("STEP 8: Checking Logout button...");
WebElementlogoutButton = wait.until(
ExpectedConditions.visibilityOfElementLocated(
By.xpath("//a[text()='Logout']")
                )
        );
Assert.assertTrue(
logoutButton.isDisplayed(),
                "Logout button is not visible."
        );
System.out.println("PASS: Logout button is visible.");
pause();
        // FINAL RESULT
System.out.println();
System.out.println("========================================");
System.out.println("          LOGIN AUTOMATION TEST PASSED");
System.out.println("========================================");
       // Keep browser open for another 5 seconds
System.out.println("Keeping browser open for 5 seconds...");
       try {
Thread.sleep(5000);
       } catch (InterruptedException e) {
Thread.currentThread().interrupt();
          }
      }
      @AfterMethod
   public void tearDown() {
System.out.println("Closing browser...");
       if (driver != null) { driver.quit();
          }
System.out.println("Browser closed.");
      }
  }
```

Save the file

Run the file as Testng and see the output

Run As → TestNG Test

5. Expected Console Output

You should see something similar to:

URL Validation Passed

Current URL: https://opensource-demo.orangehrmlive.com/web/index.php/dashboard/index

Dashboard Validation Passed Logout/User

Menu Validation Passed Logout option is

visible

Logout Successful

PASSED: loginTest

6. Test Cases for Your Practical

You can document the practical using these test cases:

TC ID Test Case	**Expected Result**
TC01 Open login page	Login page displayed
TC02 Enter valid username	Username entered successfully
TC03 Enter valid password	Password entered successfully
TC04 Click Login	User redirected to dashboard
TC05 Validate URL	Dashboard URL displayed
TC06 Validate welcome/dashboard message	Dashboard displayed

PRACTICAL 9

Aim: Web Element Interaction and Synchronization Handling

Prequisites: Selenium WebDriver with Java, Maven and TestNG

Elements covered: Textbox, Radio button, Checkbox, Dropdown, Alert, Frame, Implicit wait, Explicit wait

STEP 1 — Open Eclipse

Open Eclipse Then:**File → New → Maven Project**If Maven Project is not visible:
File → New → Other → Maven → Maven Project Click Next.

STEP 2 — Select Maven Project

You may see: **Create a simple project (skip archetype selection)**For an easy setup, check this box. Click Next.

STEP 3 — Enter Maven

Details Enter:

Field	Value
Group Id	com.selenium
Artifact Id	SeleniumTest
Version	0.0.1-SNAPSHOT
Packaging	jar

Then click Finish. **Your project should look approximately like:**

selenium-test

├── src/main/java
├── src/test/java
├── JRE System Library
├── Maven Dependencies

STEP 4 — Open pom.xml Open: Paste code and save and update maven even update main project

```
<?xml version="1.0" encoding="UTF-8"?>
  <project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
      <modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.selenium</groupId>
<artifactId>WebElementPractical</artifactId>
<version>0.0.1-SNAPSHOT</version>
<properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
<dependencies>
    <!-- Selenium WebDriver -->
    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>4.35.0</version>
    </dependency>
    <!-- TestNG -->
    <dependency>
        <groupId>org.testng</groupId>
        <artifactId>testng</artifactId>
        <version>7.11.0</version>
        <scope>test</scope>
      </dependency>
  </dependencies>
<build>
    <plugins>
        <!-- Maven Compiler Plugin -->
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.13.0</version>
            <configuration>
                <source>17</source>
                <target>17</target>
            </configuration>
        </plugin>
        <!-- Maven Surefire Plugin -->
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.5.0</version>
            <configuration>
                <suiteXmlFiles>
                    <suiteXmlFile>testng.xml</suiteXmlFile>
                </suiteXmlFiles>
            </configuration>
        </plugin>
    </plugins>
 </build>
</project>
```

STEP 5 — Create the Java Package

Go to:src/test/javaRight-click it.Select:New → Package/ Enter: **com.selenium.test**

STEP 6 — Create Java Class

Right-click:**com.selenium.test** and Select:New → Class and Enter:

WebElementInteractionTest and Click Finish.

STEP 7 — Paste the Complete Java Code

```
package com.selenium.test; import java.time.Duration;
import org.openqa.selenium.Alert;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver; import
org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions; import
org.openqa.selenium.support.ui.WebDriverWait; import
org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
  public class WebElementInteractionTest {
     WebDriver driver;
     WebDriverWait wait;
     @BeforeMethod public
     void setUp() {
        System.out.println("========================================");
        System.out.println("STEP 1: Starting Chrome browser.");
        System.out.println("========================================");
        driver = new ChromeDriver();
        driver.manage().window().maximize();
        driver.manage().timeouts().implicitlyWait(
              Duration.ofSeconds(10));
        wait = new WebDriverWait(
              driver,
              Duration.ofSeconds(15));
        System.out.println("Browser started successfully.");
        waitForSeconds(2);
     }
     @Test
     public void testWebElements() {
        System.out.println();
        System.out.println("========================================");
        System.out.println("STEP 2: Opening DemoQA Practice Form.");
        System.out.println("========================================");
```

```
driver.get("https://demoqa.com/automation-practice-form");
System.out.println("Practice form opened.");
waitForSeconds(3);
System.out.println();
System.out.println("========================================");
System.out.println("STEP 3: Entering First Name.");
System.out.println("========================================");
WebElement firstName = wait.until(
        ExpectedConditions.visibilityOfElementLocated(
                By.id("firstName")));
firstName.clear();
firstName.sendKeys("Bhupinder");
System.out.println("First Name entered: Bhupinder");
waitForSeconds(2);
System.out.println();
System.out.println("========================================");
System.out.println("STEP 4: Entering Last Name.");
System.out.println("========================================");
WebElement lastName = wait.until(
        ExpectedConditions.visibilityOfElementLocated(
                By.id("lastName")));
lastName.clear();
lastName.sendKeys("Singh");
System.out.println("Last Name entered: Singh");
waitForSeconds(2);
System.out.println();
System.out.println("========================================");
System.out.println("STEP 5: Entering Email.");
System.out.println("========================================");
WebElement email = wait.until(
        ExpectedConditions.visibilityOfElementLocated(
                By.id("userEmail"))); email.clear();
email.sendKeys("bhupinder\\@gmail.com");
System.out.println(
        "Email entered: bhupinder\\@gmail.com");
waitForSeconds(2);
System.out.println();
System.out.println("========================================");
System.out.println("STEP 6: Selecting Male radio button.");
System.out.println("========================================");
WebElement maleRadio = wait.until(
        ExpectedConditions.elementToBeClickable(
                By.xpath("//label[text()='Male']")));
maleRadio.click();
System.out.println("Male radio button selected.");
```

```
waitForSeconds(2);
System.out.println();
System.out.println("========================================");
System.out.println("STEP 7: Selecting Sports checkbox.");
System.out.println("========================================");
WebElement sportsCheckbox = wait.until(
        ExpectedConditions.elementToBeClickable(
                By.xpath("//label[text()='Sports']")));
sportsCheckbox.click(); System.out.println("Sports
checkbox selected."); waitForSeconds(3);
System.out.println();
System.out.println("========================================");
System.out.println("STEP 8: Selecting State dropdown.");
System.out.println("========================================");
WebElement stateDropdown = wait.until(
        ExpectedConditions.elementToBeClickable(
                By.id("state"))); stateDropdown.click();
System.out.println("State dropdown opened.");
waitForSeconds(2);
WebElement ncrOption = wait.until(
        ExpectedConditions.elementToBeClickable(
                By.xpath(
                        "//div[contains(@id,'react-select-3-option-0')]")));
ncrOption.click();
System.out.println("State selected: NCR");
waitForSeconds(2);
System.out.println();
System.out.println("========================================");
System.out.println("STEP 9: Selecting City dropdown.");
System.out.println("========================================");
WebElement cityDropdown = wait.until(
        ExpectedConditions.elementToBeClickable(
                By.id("city"))); cityDropdown.click();
System.out.println("City dropdown opened.");
waitForSeconds(2);
WebElement delhiOption = wait.until(
        ExpectedConditions.elementToBeClickable(
                By.xpath(
                        "//div[contains(@id,'react-select-4-option-0')]")));
delhiOption.click();
System.out.println("City selected: Delhi");
waitForSeconds(2);
System.out.println();
```

```
System.out.println("========================================");
System.out.println("STEP 10: Opening Alerts page.");
System.out.println("========================================");
driver.get("https://demoqa.com/alerts");
waitForSeconds(3); System.out.println("Alerts
page opened."); System.out.println();
System.out.println("========================================");
System.out.println("STEP 11: Clicking alert button.");
System.out.println("========================================");
WebElement alertButton = wait.until(
        ExpectedConditions.elementToBeClickable(
                By.id("alertButton")));
alertButton.click();
System.out.println("Alert displayed.");
waitForSeconds(2); System.out.println();
System.out.println("========================================");
System.out.println("STEP 12: Reading and accepting alert.");
System.out.println("========================================");
Alert alert = wait.until(
        ExpectedConditions.alertIsPresent());
String alertText = alert.getText();
System.out.println(
        "Alert message: " + alertText);
alert.accept(); System.out.println("Alert
accepted."); waitForSeconds(3);
System.out.println();
System.out.println("========================================");
System.out.println("STEP 13: Working with frame.");
System.out.println("========================================");
driver.get("https://demoqa.com/frames");
waitForSeconds(3);
wait.until(
        ExpectedConditions.frameToBeAvailableAndSwitchToIt(
                By.id("frame1")));
System.out.println(
        "Successfully switched into frame.");
WebElement frameHeading = wait.until(
        ExpectedConditions.visibilityOfElementLocated(
                By.id("sampleHeading")));
String frameText = frameHeading.getText();
System.out.println(
        "Text inside frame: " + frameText);
waitForSeconds(4);
```

```
        driver.switchTo().defaultContent();
        System.out.println(
                "Returned to main page.");
        waitForSeconds(3); System.out.println();
        System.out.println("========================================");
        System.out.println("        TEST COMPLETED SUCCESSFULLY");
        System.out.println("========================================");
        waitForSeconds(5);
    }
    public void waitForSeconds(int seconds) { try
        {
            Thread.sleep(seconds * 1000);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            System.out.println(
                    "Wait interrupted.");
        }
    }
    @AfterMethod
    public void tearDown() {
        System.out.println();
        System.out.println("========================================");
        System.out.println(
                "Waiting 5 seconds before closing browser...");
        System.out.println("========================================");
        waitForSeconds(5);
        if (driver != null) {
            driver.quit();
            System.out.println(
                    "Browser closed successfully.");
        }
    }
}
```

Select the code and run as TestNg:

```
========================================
STEP 1: Starting Chrome browser.
========================================
Oct 08, 2026 10:59:55 PM org.openqa.seleniu
WARNING: Unable to find CDP implementation
Oct 08, 2026 10:59:55 PM org.openqa.seleniu
WARNING: Unable to find version of CDP to u
Browser started successfully.

========================================
STEP 2: Opening DemoQA Practice Form.
========================================
Practice form opened.

========================================
STEP 3: Entering First Name.
========================================
First Name entered: Bhupinder

========================================
STEP 4: Entering Last Name.
========================================
Last Name entered: Singh

========================================
STEP 5: Entering Email.
========================================
Email entered: bhupinder@gmail.com

========================================
STEP 6: Selecting Male radio button.
========================================
Male radio button selected.

========================================
STEP 7: Selecting Sports checkbox.
========================================
Sports checkbox selected.

========================================
STEP 8: Selecting State dropdown.
========================================

========================================
Waiting 5 seconds before closing browser...
========================================
Browser closed successfully
```

```
===============================================
    Default test
    Tests run: 1, Failures: 1, Skips: 0
===============================================


===============================================
Default suite
Total tests run: 1, Passes: 0, Failures: 1, Skips: 0
===============================================
```

CONCLUSION:

Thus, the Selenium WebDriver practical was successfully performed using Java, Maven, and TestNG. Various web elements such as text boxes, radio buttons, checkboxes, dropdowns, alerts, and frames were successfully identified and interacted with. Implicit and explicit waits were also implemented to handle synchronization between the WebDriver and web page elements. Hence, the practical helped in understanding web element interaction and synchronizati

PRACTICAL 10

Aim: Load testing using appache Jmeter_

Design and execute load testing scenarios for web applications and analyse response time.

- Software Required
- Apache JMeter
- Java JDK
- Web browser
- Internet connection

PROCEDURE

1. Download and install Apache JMeter.
    1. Open JMeter and create a new Test Plan.
    2. Add a **Thread Group** to the Test Plan.
    3. Configure the number of virtual users, ramp-up period, and loop count.
    4. Add an **HTTP Request** sampler and enter the target server name and path.
    5. Add a **Listener**, such as View Results Tree or Summary Report.
    6. Run the test plan.
    7. Analyse the response time, throughput, and error rate from the results.

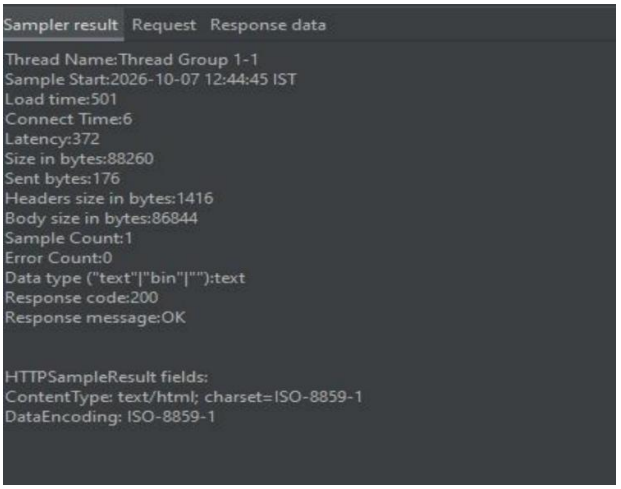
Expected Output

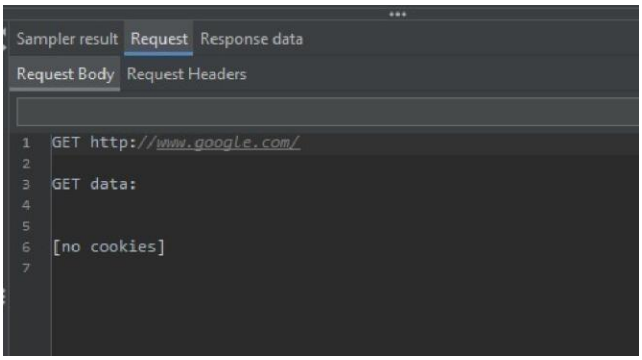
The JMeter results display the following performance metrics:

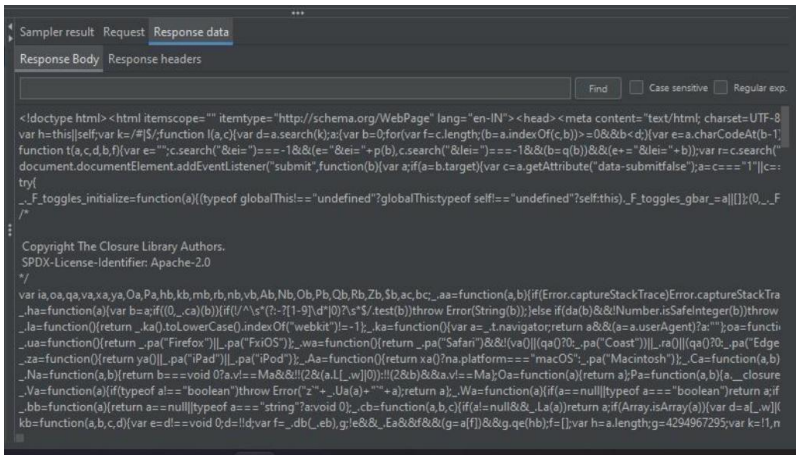
- **Sample Time:** Time taken to receive a response.
- **Average Response Time:** Average time taken to process requests.
- **Throughput:** Number of requests processed per unit of time.
- **Error Percentage:** Percentage of failed requests.
- **Min/Max Response Time:** Minimum and maximum response times recorded.

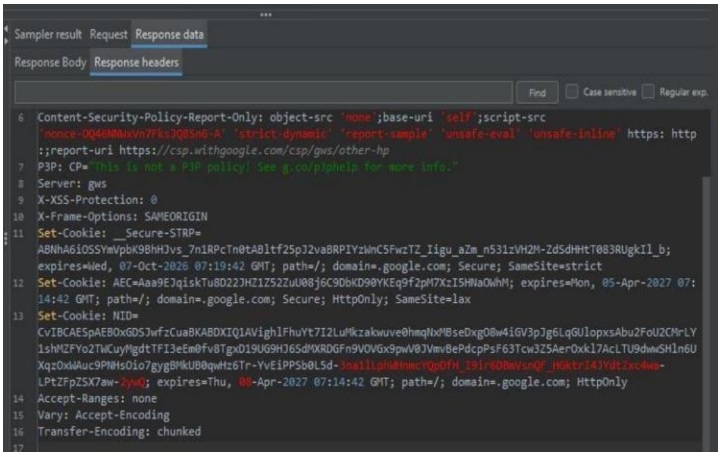
Note: Actual response times and error percentages depend on the target server and network conditions.

Result:-


Sampler result
Request
Response data
Thread Name:Thread Group 1-1
Sample Start:2026-10-07 12:44:45 IST
Load time:501
Connect Time:6
Latency:372
Size in bytes:88260
Sent bytes:176
Headers size in bytes:1416
Body size in bytes:86844
Sample Count:1
Error Count:0
Data type ("text"|"bin"|""):text
Response code:200
Response message:OK
HTTPSampleResult fields:
ContentType: text/html; charset=ISO-8859-1
DataEncoding: ISO-8859-1







CONCLUSION:

Load testing using Apache JMeter was performed to simulate multiple users accessing a web application. The response time, throughput, and error percentage were analysed to evaluate the application's performance under load.

Apache JMeter is an open-source testing tool used to perform load testing, performance testing, and stress testing of web applications. It simulates multiple virtual users accessing an application simultaneously and measures performance metrics such as response time, throughput, and error percentage.